

SWE-bench Multilingual

300 curated SWE-bench style tasks from 42 repositories representing 9 programming languages

Kabir Khandpur, Kilian Lieret, Carlos E. Jimenez, Ofir Press, John Yang

GitHub | Dataset

Originally posted as a blog post on Kabir's website.

Summary

This post introduces SWE-bench Multilingual, a new benchmark in the SWE-bench family designed to evaluate the software engineering capabilities of LLMs across a range of programming languages. SWE-bench Multilingual consists of 300 curated software engineering tasks derived from real-world GitHub pull requests across 42 repositories and 9 programming languages. The repositories span a wide range of application domains, including web frameworks, data storage and processing tools, core utilities, and common libraries.

Using the SWE-agent framework, **Claude 3.7 Sonnet achieves a 43% resolution rate on SWE-bench Multilingual**, compared to 63% on SWE-bench Verified, highlighting room for improvement in languages other than Python.

The dataset is available on HuggingFace with evaluation code integrated into the SWE-bench repository. Please email me with any questions or feedback!

Introduction

SWE-bench is a standard benchmark to evaluate LLMs on software engineering capabilities. The benchmark dataset consists of 500 GitHub issues from 17 different Python projects. Given its focus on Python, the benchmark is not representative of LLM performance in different programming languages and domains. To broaden SWE-bench's evaluation capability, I developed SWE-bench Multilingual in collaboration with the SWE-bench team to accomplish the following goals:

2. Remain fully compatible with SWE-bench, so existing users can easily evaluate on Multilingual without needing to change their infrastructure.
3. Create a high-quality dataset that is comprehensive but small enough to run quickly. While concurrent work like Multi-SWE-bench provides more task instances in multiple languages, this dataset is purposely limited to 300 high-quality tasks so that the evaluation is easy to run quickly.

To achieve these goals, the Multilingual dataset provides 300 tasks across 42 repositories and 9 popular programming languages: C, C++, Go, Java, JavaScript, TypeScript, PHP, Ruby and Rust. See Appendix A for dataset details.

Benchmark construction

SWE-bench Multilingual follows the same collection strategy, dataset format and evaluation protocol as SWE-bench. Task instances correspond to real-world GitHub issues and their resolving pull requests (PRs). An agent receives the issue description and a repository snapshot at the pre-solution state and generates code modifications that resolve the issue. Success is determined by passing two sets of unit tests derived from the original PR: fail-to-pass (F2P) tests, which ensure the specific issue is fixed, and pass-to-pass (P2P) tests, which verify that existing functionality remains intact.

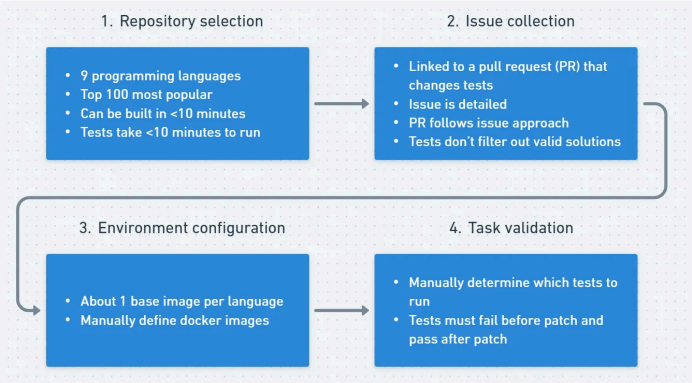


Figure 1. Collection pipeline

1. Repository selection

First, 9 popular programming languages are selected based on the annual Stack Overflow Developer Survey. Then, from the top 100 most starred repositories in these languages are selected from GitHub-Ranking, repositories where the primary language is English and there are a large number of candidate issues are selected. The public contribution guidelines and GitHub action workflow files are used to determine the correct commands to build the codebase and run

2. Issue collection

For each repository, the SWE-bench collection pipeline is used to collect issue-PR pairs where the PR contains at least one test file. An issue is rejected if it doesn't describe the problem in sufficient detail, if the corresponding pull request implemented a different approach than the one proposed in the issue, if the pull request contains changes related to multiple issues, and if tests filter out valid solutions (for example, by checking for a specific error message).

3. Environment configuration

The original SWE-bench repository defines three layers of docker images:

1. A base image that installs the language runtime and operating system packages for all the repositories in that language.
2. An environment image that installs the conda and pip packages required to run tests for a particular task instance. This image acts as a cache to avoid re-downloading dependencies between test runs.
3. An instance image that runs the test. For each instance, the install and test commands are manually specified.

Multilingual provides base and instance images but opts to skip defining environment images. While environment images worked well for SWE-bench, given its numerous tasks from a few repositories sharing dependencies, Multilingual's 300 tasks across 42 repositories rarely share dependencies, making pre-built environment images less beneficial and significantly increasing manual curation effort compared to the original SWE-bench.

4. Task validation

A final manual verification procedure is conducted before including a task instance in the dataset.

1. Clone the repository and check out the commit specified by the issue.
2. Run instance-specific pre-install and install commands.
3. Run relevant tests, including tests added by the pull request. If tests added by the pull request pass without any code changes, the task instance is discarded.
4. Apply the test files.
5. Build the codebase.
6. Apply the “gold” patch, i.e. the non-test code changes introduced by the pull request.
7. Run relevant tests again and verify that they pass.
8. Parse the test logs to get the list of tests passing and failing tests for inclusion in the dataset.

To establish baseline performance, I evaluated SWE-agent + Claude 3.7 Sonnet on the Multilingual dataset. With a cost limit of $2.50, this setup correctly resolves 43% of tasks.

Resolution rate varies by language (figure 2), with Rust having the highest resolution rate and C/C++ the lowest.

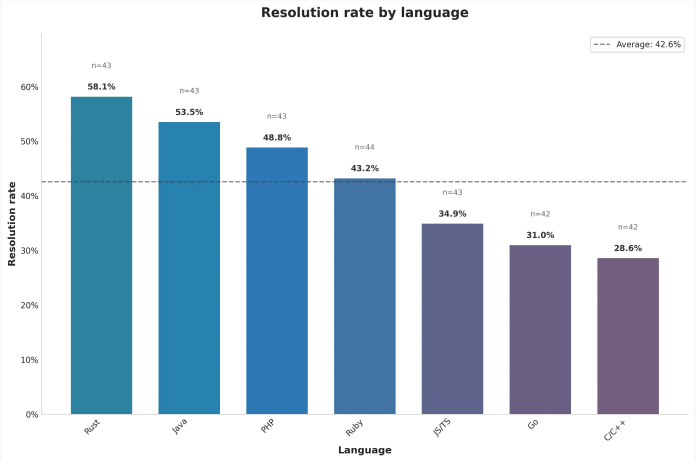


Figure 2. Resolution rate by language

It's possible that the difference in resolution rate is because the dataset happens to contain more difficult tasks in some languages. In the absence of human annotations of difficulty, the number of lines of code modified by the gold patch can be used as an estimate of task difficulty. Figure 3 shows that difficulty distribution within a language isn't obviously correlated with resolution rate. For example, solutions to Rust tasks modify more lines of code on average, yet SWE-agent had the highest resolution rate in Rust.

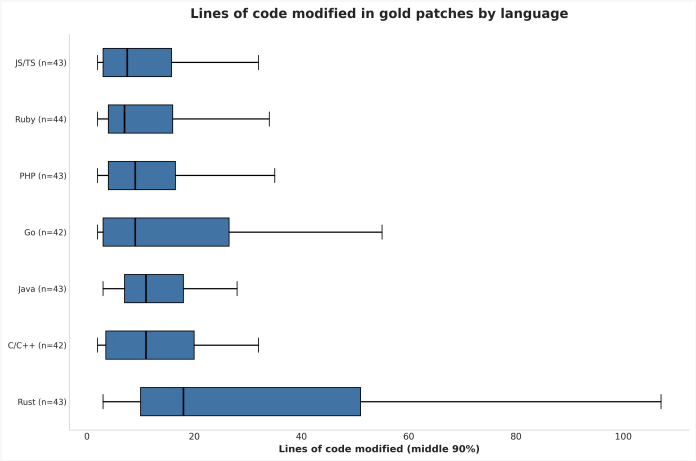


Figure 3. Lines of code updated by language

From this limited analysis, it's hard to say what factors are most important in determining resolution rate, though it looks like language and difficulty are both relevant. Appendix B contains more cuts of the resolution rate statistics.

successful and failed tasks (see Appendix C for more notes). The distribution of the actions that SWE-agent took was very similar between successful and failed tasks (figure 5), suggesting that model capabilities are the limiting factor in resolution rate rather then agent design.

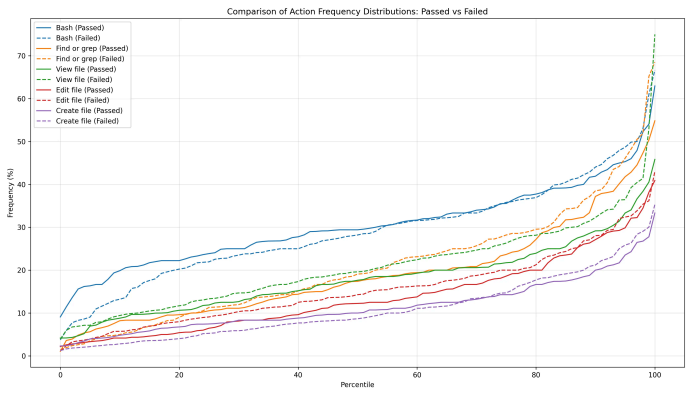


Figure 5. Action frequency distribution in successful vs. failed tasks. Each action type has two lines - a solid line for successful tasks and a dashed line of the same color for failed ones. Every solid and dashed line pair are similar, indicating actions that are used similarly between successful and failed attempts.

Limitations

Most tasks require only a few lines of code. SWE-bench's collection strategy looks for issues where the problem is well-defined and unit tests are unambiguous. This naturally selects for pull requests where only a few lines of code are modified. In SWE-bench Multilingual, the gold patch modifies 10 lines of code at the median and 110 lines at the 95th percentile. Real-world software engineering tasks often require significantly larger code modifications.

Only one model is evaluated. Due to budget constraints, only Claude 3.7 Sonnet was evaluated. Other models or agent frameworks may score higher on this dataset, possibly comparable in score to their performance on SWE-bench Verified.

No human annotations. The dataset is manually curated, but no human annotations of difficulty are included, as is done in SWE-bench Verified and Multi-SWE-bench.

Conclusion

SWE-bench Multilingual suggests that LLMs are more proficient in Python than other languages. The dataset is available on HuggingFace with evaluation code integrated into the SWE-bench

Appendix A: Dataset

Overview

Repository	Language	Issue C
redis/redis	C	
jqlang/jq	C	
nlohmann/json	C++	
micropython/micropython	C	
valkey-io/valkey	C	

Median values

Repository	Issue text	Gold p
	Word count	Lines
apache/druid	165	11
apache/lucene	135	7
astral-sh/ruff	134	14
axios/axios	197	5

Appendix B: Evaluation results

Resolution rate by repository

micropython/micropython	C	0	5
babel/babel	JS/TS	0	5
faker-ruby/faker	Ruby	0	2
caddyserver/caddy	Go	2	12
briannesbitt/carbon	PHP	2	8

Resolution rate by language

Language	Resolved	Unresolved	Total
C/C++	12	30	42
Go	13	29	42
JavaScript/TypeScript	15	28	43
Ruby	19	25	44
PHP	21	22	43

Resolution rate by year

Year	Resolved	Unresolved	Resolution rate
≤2021	16	22	42.1%
2022	24	30	44.4%
2023	27	48	36.0%
2024	54	63	46.2%

2025	7	9	43.8%

Appendix C: Miscellaneous notes

Issue selection

During the collection process, I needed to inspect a large number of candidate issues for inclusion in the dataset. I wrote a web app to make it easier to determine whether issues were unsuitable by showing the issue and pull request description side by side. The app also let me keep track of the issues I'd already looked at.

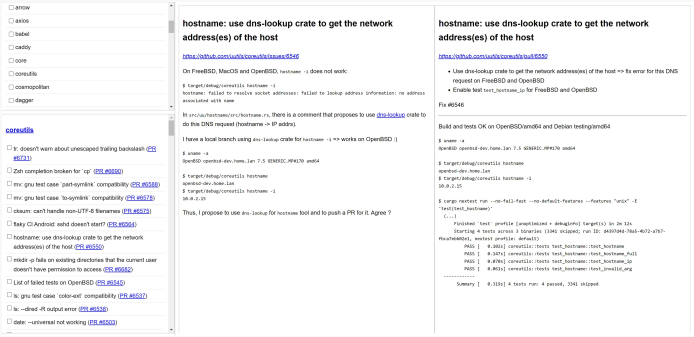


Figure 6. Issue inspection app

Agent trajectory inspection

To help look for patterns in the trajectories of successful and failed tasks, I created a web app that displays the actions taken by the SWE-agent in a task. A chat pane on the side let me ask questions about the trajectory. Unfortunately, I wasn't able to find any patterns in the trajectories.

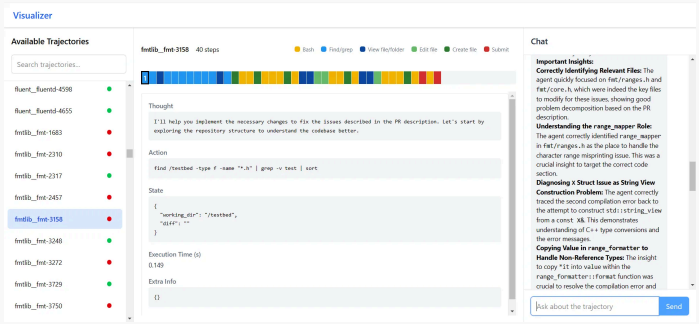


Figure 7. Agent trajectory inspection app

Nonetheless, I found the pattern of visual inspection to be very useful and a perfect use case for vibe coding. I'll continue to make small user interfaces like this in future projects.

Notes for agent improvements

include any such issues, but future agents should have this capability.

Support multiple languages. As the SWE-bench Multimodal paper notes, many open-source agent frameworks hardcode Python support.

> …except for SWE-agent, the systems that we study (Agentless, Moatless, and AutoCodeRover) impose fixed, procedural problem-solving workflows. Every system starts with a bug localization step that relies on abstract syntax tree (AST) parsing libraries to identify programmatic symbols.

Correspondence

For questions about SWE-bench Multilingual, please contact:

{kabir.khandpur@gmail.com,johnby@stanford.edu}

Citation

If you use SWE-bench Multilingual in your research, please cite the SWE-smith paper:

BibTeX | APA | MLA

Copy

```
@misc{yang2025swesmith,
  title={SWE-smith: Scaling Data for Software Engineering Agents},
  author={John Yang and Kilian Lieret and Carlos E. Jimenez and Alexander Wettig and Kabir Khandpur and Yanzhe Zhang and Binyuan Hui and Ofir Press and Ludwig Schmidt and Diyi Yang},
  year={2025},
  eprint={2504.21798},
  archivePrefix={arXiv},
  primaryClass={cs.SE},
  url={https://arxiv.org/abs/2504.21798},
}
```

© 2026 SWE-bench Team. All rights reserved.



SWE-bench